Data Consistency Across Systems

write once, propagate reliably, and reconcile anyway

Watch the 4-min video

In one line: never write the same change separately to several systems (dual writes); write it once to the source of truth, propagate it with a **transactional outbox** or **change data capture** into **idempotent, version-aware** sinks, and run **reconciliation** jobs that find and repair the drift that still happens.

1 Every system a little different

An order is in the database but not in search. The cache shows the old price. The warehouse says yesterday's revenue was $12,000; the payment provider says $13,000. Each copy (index, cache, warehouse, read model, partner system) is there for a good reason, but every copy can disagree with the source, and customers notice. Keeping copies aligned is a discipline with three parts: **propagate** reliably, **apply** safely, **verify** continuously.

2 The trap: dual writes

The tempting code writes to the database, then the search index, then the cache, then publishes to a queue. There is no transaction across those systems, so:

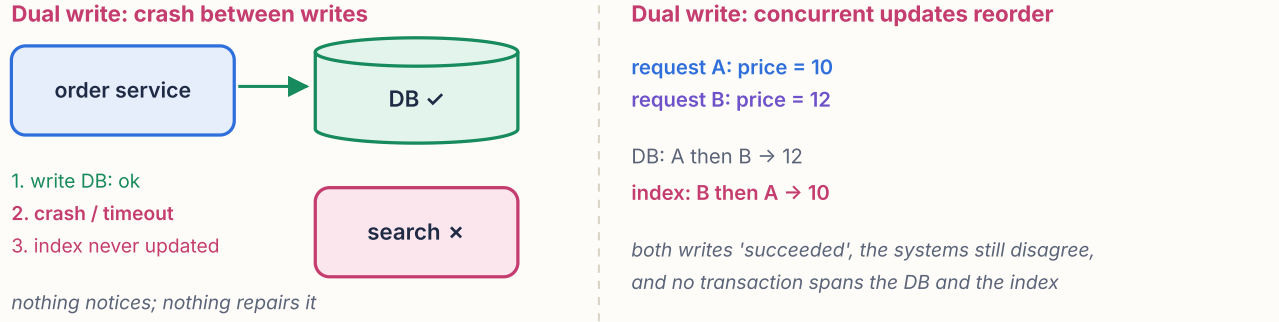


Figure 1. Two ways dual writes go wrong: a partial failure, and a race that reorders updates. Both leave permanent, silent drift.

- A crash, timeout or error between writes leaves some systems updated and others not, and nothing remembers to finish the job.
- Retrying the whole sequence risks duplicates; not retrying risks loss.
- Concurrent requests can apply updates in a different order in each system.
- Swapping the order (queue first, then DB) just moves the problem: now consumers can see an event for a change that was rolled back.

Dual writes are the most common source of drift precisely because they look simple in code.

3 Write once, propagate from the source

Transactional outbox. In the same local transaction as the business change, insert a row into an `outbox` table describing the event. Either both commit or neither. A **relay** then reads unpublished outbox rows (by

polling, or by CDC on the outbox table) and publishes them to the broker, marking or deleting them afterwards. Publishing is **at-least-once**: a relay crash after publish but before marking repeats the event, so consumers must be idempotent.

Change data capture (CDC). Read the database's own change log (PostgreSQL WAL via logical decoding, MySQL binlog, SQL Server CDC, MongoDB change streams) and publish every committed change. Debezium is the common open-source tool; each event carries the row before and after, the operation, and the log position. No application code changes are needed, but events mirror table rows, not business meaning, and they couple consumers to your schema.

	Transactional outbox	CDC on business tables
Event shape	designed business events (`OrderPaid`)	row changes (before / after)
Code change	write the outbox row in each transaction	none in the app
Schema coupling	low: event is a contract	high: consumers see table columns
Captures manual SQL fixes	no	yes, every committed change
Typical use	service-to-service events	replication to search, cache, warehouse

Many teams combine them: the app writes an outbox row, and Debezium's outbox event router streams the outbox table, giving designed events without a polling relay. Operational gotcha: a PostgreSQL replication slot that no consumer reads keeps all WAL on disk, and can fill it. Monitor slot lag.

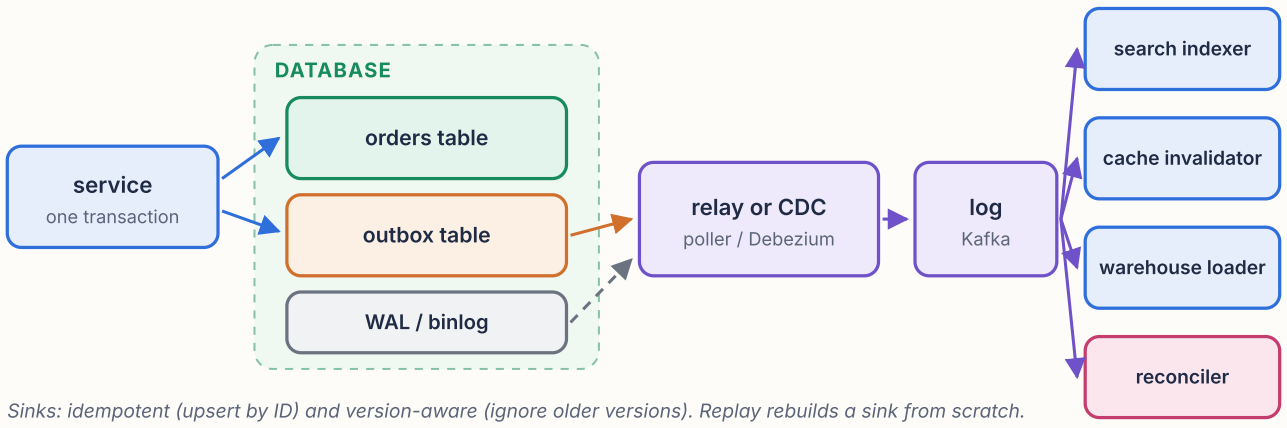


Figure 2. One write, many propagated copies. The log (Kafka) lets each sink consume at its own pace and lets a new sink replay history.

4 Idempotent, version-aware sinks

Events can arrive **twice** (at-least-once delivery) and, across partitions or retries, **out of order**. Each sink must handle both:

- **Idempotent**: key every write by the source record's ID (upsert, not insert), so applying the same event twice gives the same result.
- **Version-aware**: carry a monotonically increasing version (a row version column, or the CDC log position) and apply an update only if it's newer than what the sink holds. Elasticsearch supports this natively with `version_type=external`; SQL sinks use a conditional upsert (below).

- **Ordering by key**: partition the log by record ID so all changes to one record go through one partition in order.
- **Caches**: prefer deleting the key on change over writing the new value, so a late, stale event can't put old data back.

The reward: you can **replay** the log, or re-snapshot the source, to rebuild a sink from scratch at any time, which is also the repair tool of last resort.

```
BEGIN;
  UPDATE orders SET status = 'paid', version = version + 1 WHERE id = 42;
  INSERT INTO outbox (id, aggregate_id, type, payload)
    VALUES (gen_random_uuid(), 42, 'OrderPaid', '{"version": 7, ...}');
COMMIT;   -- both rows or neither

-- sink: version-aware upsert (older events are ignored)
INSERT INTO order_view (id, status, version) VALUES (42, 'paid', 7)
ON CONFLICT (id) DO UPDATE SET status = EXCLUDED.status, version = EXCLUDED.version
  WHERE order_view.version < EXCLUDED.version;
```

Figure 3. The outbox write and a version-aware sink upsert. A replayed or late event with version ≤ 7 changes nothing.

5 Reconciliation: verify, then repair

Even good pipelines drift: a bug in a consumer, a skipped event, a manual fix in one system, a restore of an old backup. A **reconciliation job** compares systems for a period and repairs differences, so drift is short-lived instead of permanent:

- **Cheap first**: compare counts and sums per bucket (per day, per tenant, per ID range).
- **Then drill down**: compute a checksum per bucket (hash of IDs and versions); only mismatching buckets are compared row by row. This is the same anti-entropy idea databases use with Merkle trees.
- **Repair** by re-publishing the source's events for those records, or upserting the target directly, and **record** every repair. A rising repair count is an alert that a pipeline is broken.

6 Reconciling money

For payments, your **ledger** and the provider's **settlement report** must match to the cent. The process is formal: extract both sides, normalise (currency, fees, time zone of the provider's day boundary), match on a shared key (payment or charge ID, amount, currency), and classify the rest as exceptions. Some exceptions are fixed automatically (a lost webhook: fetch the payment from the provider's API and record it); others (amount mismatch, unexplained refunds) go to a person with an SLA. Mature teams add a third leg: the bank statement showing the payout actually arrived.

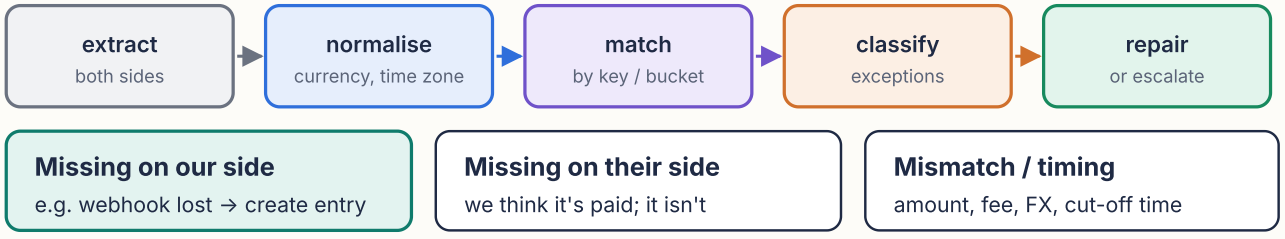


Figure 4. A reconciliation pipeline and its exception classes. Timing differences (a payment at 23:59 in one system's day and 00:01 in the other's) are the most common false alarm.

WORKED EXAMPLE: THE NIGHTLY LEDGER CHECK

At 02:00 the job downloads yesterday's settlement file from the payment provider and sums the internal ledger by **currency and merchant**, using the provider's UTC day boundary. Totals match for every merchant except one, which is short by $412.50. Drilling down by payment ID finds **three captured payments** that exist in the provider's file but not in the ledger: their webhooks never arrived (the endpoint returned errors during a deploy, and retries were exhausted). The job fetches each payment from the provider's API to confirm it, creates the missing ledger entries through the normal service (so the outbox emits events and downstream systems update too), and records the repair with a reason code. A dashboard shows zero open exceptions by 02:15; the lost-webhook count raises a ticket to fix the endpoint's retry handling.

COMMON MISTAKES

- Writing to DB, index and queue in sequence and calling it "eventually consistent".
- Sinks that insert instead of upsert, creating duplicates on replay.
- Last-write-wins by arrival time instead of by source version.
- Repairing a sink by hand, so the source and the log never learn about it.

7 Key terms

Dual write: updating several systems separately for one change

CDC: streaming committed changes from the database log

Version-aware: older updates never overwrite newer ones

Transactional outbox: event row written in the same transaction as the change

Idempotent sink: applying an event twice has the same effect as once

Reconciliation: comparing systems and repairing differences

8 Interview questions

Q1. How do you detect and repair data drift between systems? BASIC

Avoid dual writes; write to one source of truth and propagate changes with an outbox or CDC into idempotent, version-aware sinks; then run reconciliation jobs comparing counts, sums or checksums and repair the differences.

Q2. Why are dual writes dangerous? BASIC

A failure or crash between writes leaves systems inconsistent with nothing to repair it, and concurrent writes can be applied in different orders in different systems.

Q3. Outbox or CDC? BASIC

Outbox for designed business events between services; CDC for replicating table changes to search, caches and warehouses with no app changes; they are often combined.

Q4. Search sometimes shows an old price after updates. Why, and how do you fix it? SCENARIO

Out-of-order or duplicate events overwrite newer data; partition by product ID and make the indexer version-aware (external versioning), so older versions are rejected.

Q5. How would you reconcile two tables with 500 million rows each? SCENARIO

Bucket by ID range or date, compare counts and checksums per bucket, and only compare rows inside mismatching buckets; repair through the normal propagation path.

Q6. Why not publish the event first, then write the database? TRICKY

If the DB write fails or rolls back, consumers have already acted on a change that never happened; the outbox ties both to one transaction.

Q7. Your CDC connector was down for two days and the database disk filled up. Why? TRICKY

Its replication slot kept all WAL since the last confirmed position; monitor slot lag and drop or advance abandoned slots.

Next: SD-V-017 · Privacy Engineering